

A-Evolve: Our Shared Mental Model

Self-Evolving Agents with Frontier Models as Researchers

Hanqing Lu — A-EVO Lab — May 2026 (5-minute version)

A-Evolve treats self-evolution as *AI-as-researcher*: frontier agents and models play the researcher role in the loop that improves agent capability — reading trajectories, proposing modifications, running experiments, and updating either the agent’s *harness* or its *model weights*. The design space has clarified into **three categories of evolver**, each tied to a distinct task feedback signature: *A-Evolve-Agent* (harness on offline agentic tasks), *A-Evolve-Adaptive* (harness on long-running heterogeneous streams), *A-Evolve-Training* (model-weight post-training). All three now carry load-bearing results; the strongest is the first known autonomous match of a human post-training team at 30B. Thesis: *AI-as-researcher generalizes from external scaffold to model parameters*.

Where we stand (one evolver across all, solver fully automated). **Training:** Nemotron-30B post-training **0.86 (rank 8/4000, vs. human #1 0.87)**. **Agentic harness:** MCP-Atlas **#1 (79.4%)** — SkillBench **#2 (34.9%)** — SWE-bench Verified **#5 (76.8%)** — Terminal-Bench 2.0 **#7 (76.5%)** — ARC-AGI **+2.2pp** — CL-Bench **+4.5pp** — τ -bench **+4.3pp** — WebArena-Infinity **+3.8pp**. **Long-running, open-ended agentic streams:** PolyBench (prediction markets) Accuracy **80.9%** vs. next-best 50.8% — CTF-Dojo (security) Pass **50.2%** vs. 45.2% — FutureX (forecasting) Pass **49.5%** vs. 47.5%.

1. Thesis

AI agents and models themselves can play the researcher role in the loop that improves agent capability — reading trajectories, proposing modifications, running experiments, deciding what to try next. This became possible only in the last ~ 12 months, once frontier base models became strong enough to do researcher tasks end-to-end. It has now cleared a load-bearing test in all three evolver categories (§3–§5). The practical consequence: agent self-improvement is bottlenecked not by hand-designed pipelines or curated data, but by **how much researcher capacity we can compose inside the loop** — and (§6) by how long that loop can stay coherent.

2. Scope and sequencing: why this categorization, and why this order

AI research itself decomposes into four parts: **hypothesis generation**, **experiment execution**, **strategy** (budget across hypotheses, which signals to attend to), and **infra** (the substrate experiments run on). The three categories below differ along this decomposition — each adds an order of magnitude of cost to one or two of these elements — and that asymmetry dictated our sequencing.

	A-Evolve-Agent	A-Evolve-Adaptive	A-Evolve-Training
<i>What is evolved</i>	harness (prompts / skills / memory / workflow)	harness + evolver state + routing tree	training config, dataset, training code
<i>Final output</i>	agent with stronger harness	agent that sustains performance on open-ended streams	model with updated weights
<i>Task shape</i>	agentic benchmarks (e.g., MCP-Atlas, SWE-bench, Terminal-Bench, ARC-AGI, τ -bench)	long-running, open-ended agentic task streams (e.g., prediction markets, security competitions, event forecasting)	end-to-end post-training run
<i>Active research elements</i>	hypothesis, execution	hypothesis, execution, strategy	hypothesis, execution, strategy, infra
<i>Papers</i>	Evolver-Solver-Bench ^[2] , Evo-Harness ^[3]	Adaptive Auto-Harness ^[4]	A-Evolve-Training ^[1]

Why this order. Going Agent $\rightarrow$ Adaptive $\rightarrow$ Training, execution and infra cost each jump roughly an order of magnitude (agent rollouts $\rightarrow$ stream-long deployments $\rightarrow$ end-to-end frontier training), and each category cumulatively activates one more research element (Adaptive adds strategy because long-running deployments force budget allocation; Training adds infra because the training config and code are themselves modified by the evolver). The first two categories also *de-risk* the third: Agent verifies that frontier models can play researcher at all; Adaptive verifies that the multi-agent / stateful / HITL machinery pays off; only then do we spend $1000\times$ compute on Training. The same lens explains why Karpathy-style auto-research scales fluently on GPT-2 (~ 630 -line training file $\Rightarrow$ infra cost ≈ 0) but does not naively transfer to frontier post-training — once execution requires actual end-to-end frontier training, every other element of the loop becomes orders harder. Framework primitives that absorb all three categories are open-source at github.com/A-EVO-Lab/a-evolve.

3. A-Evolve-Training: first end-to-end post-training at 30B reaching human parity

A-Evolve-Training^[1] runs end-to-end post-training of a 30B Nemotron model autonomously — designing data mixtures, training schedules, hyperparameter regimes, and ablation protocols — and scores **0.86 (rank 8 of 4000)** vs. the human

Kaggle #1’s 0.87. To our knowledge this is the first time an autonomous system has matched a human post-training team at this scale. Three system-design contributions enable the result: (i) a **minimal workspace abstraction** (typed artifact store + append-only trajectory log) that lets evolver and solver share state without bespoke orchestration; (ii) a **multi-agent coordination pattern** decomposing the loop into distinct researcher roles — hypothesis generation, ablation design, trajectory critique, artifact integration; (iii) an **automated mechanism for surfacing training regularities** from the evolver’s observed trajectories. **Implication for the broader community:** the bottleneck in frontier post-training is shifting from researcher capacity to evolver design.

4. A-Evolve-Agent: auto-harness reaching SOTA on agentic tasks

Evolver-Solver-Bench^[2] — **evolver and agent are distinct capabilities.** Across seven frontier and open-source models in both roles on SWE-bench Verified, MCP-Atlas, SkillsBench: harness-updating is *flat in base capability* (best-vs-worst evolver $\leq 3.1\text{pp}$; Qwen3.5-9B’s updates match Opus 4.6’s); harness-benefit is *non-monotonic* in base capability (mid-tier agents benefit most; weak agents fail by not loading the harness, load rate 0.446 vs ~ 0.96 for strong agents). *Operational consequence: don’t pay frontier prices for both halves of a self-improving stack — put the budget on the agent.*

Evo-Harness^[3] — **three factors govern whether online evolution helps.** With Opus 4.6 as solver, Evo-Harness wins every baseline on every benchmark: CL-Bench 29.5 $\rightarrow$ **34.0**, τ -bench 72.7 $\rightarrow$ **77.0**, WebArena-Infinity 72.5 $\rightarrow$ **76.3**. Ablations isolate three governing factors: *feedback grounding* (self-generated feedback actually hurts: CL-Bench drops 29.5 $\rightarrow$ 28.0), *skill abstraction level* (topic-only vs. general-only flips by benchmark), *solver-evolver alignment* (cross-model transfer works for Opus, reverses for Sonnet). Turns “online self-evolution” from a generic claim into a designable system.

5. A-Evolve-Adaptive: first successful auto-harness for long-running agents

Adaptive Auto-Harness^[4] — **evolution and adaptation are distinct gaps.** On open-ended task streams, self-evolving agents **peak early and decline** (a single dense harness overfits: e.g., evolved skills like `news_from_future.md` that lock onto early stream evidence). The paper formalizes the gap to an oracle harness as *evolution loss + adaptation loss* and addresses it with three components — a **stateful multi-agent evolver**, a **harness tree with solve-time routing**, and **scoped human-steering hooks**. Across three streams the system leads every reported metric vs. five auto-harness baselines plus the human-designed OctoTools: on PolyBench (prediction markets) **Accuracy 80.9%** vs. next-best 50.8%; on CTF-Dojo (security competitions) **Pass 50.2%** vs. 45.2%; on FutureX (event forecasting) **Pass 49.5%** vs. 47.5%. Ablations attribute each metric cluster to a specific component, so the three pieces are not redundant. Scoped human-in-the-loop emerges as a principled element of the loop, not a fallback.

6. The bottleneck: time horizon and stable goal pursuit

When we measure AI research capability directly, the answer depends **entirely** on time horizon. At the **1-hour scale** (read a paper, implement a code change, summarize a result) current frontier models crush the best human researchers; at the **1-week scale** (ask the right next question after each result, recognize a dead end, recover from a stuck state) they are not close. The bottleneck is *not* capability per token; it is the inability to **stably translate compute into scientific progress** — humans sustain coherent direction over weeks; models drift, get stuck, or chase dead ends. **The implicit scaling law for self-evolving systems lives here:** the right scaling axis is not parameters or training compute but *how long the loop can keep producing useful next questions before it spins in place or loses the goal*. This reframes our three categories on a common scale (Agent loops over hours, Adaptive over weeks, Training over weeks-to-months) and is why “peak-and-decline” in Adaptive Auto-Harness and “multi-agent role decomposition” in A-Evolve-Training are the same phenomenon from two sides — each agent holds a piece of goal state the single model cannot hold for long. Every algorithm we ship from here should be measured on its *time-horizon scaling curve*, not only on terminal benchmark performance.

7. Open questions

1. **Where on the time-horizon curve the loop saturates**, and what it would take in the next frontier model to push that ceiling out (calibrated risk-taking on ablation cost, sustained goal-tracking, recovery from stuck states).
2. **Whether the Agent / Adaptive / Training boundaries are stable.** A unified evolver that subsumes Adaptive into Agent (or Training into Adaptive) with the right scaffolding would tell us how much of the categorization is fundamental vs. artifact of current frontier-model time-horizon limits.

References. [1] **A-Evolve-Training** — Autonomous end-to-end post-training of a 30B model reaching human parity. In prep. [2] **Evolver-Solver-Bench** — Harness updating is not harness benefit: disentangling evolution capabilities in self-evolving LLM agents. Under ARR. [3] **Evo-Harness** — Context-to-harness skill compilation for self-evolving agents. Under ARR. [4] **Adaptive Auto-Harness** — Sustained self-improvement for agentic system deployment on open-ended task streams. Under ARR. [5] *Agentic Evolution is the Path to Evolving LLMs* (position paper). [arXiv:2602.00359](https://arxiv.org/abs/2602.00359).